

Q.1

Description

Q.2

Q. 2 Easy

Unanswered

Flag

Q028

EV Battery Swap Consumption Insights

Domain: Smart Mobility / EV Infrastructure

Difficulty: Easy

Marks: 10

Duration: 10-15 Minutes

Functions: 5

Tests: 6

Problem Statement

An EV battery-swap network records battery swap events across multiple stations. The operations team needs to analyze monthly energy consumption by battery chemistry and identify the battery type consuming the most energy.

Each function must be implemented independently.

Dataset

`data/battery_swaps.csv`

Columns:

`swap_id`

Dataset

data/battery_swaps.csv

Columns:

swap_id

station_id

battery_type

energy_kwh

swap_date

status

Task 1 — Load Battery Swap Data — 2 Marks

```
def load_battery_swap_data(
```

```
    spark: SparkSession,
```

```
    path: str
```

```
) → DataFrame:
```

Requirements

- read CSV with header
- infer schema

Q.1

Q.2

Requirements

- read CSV with header
- infer schema
- convert swap_date using to_date()
- return DataFrame

Task 2 — Add Swap Month Bucket — 2 Marks

```
def add_swap_month(
```

```
    df: DataFrame
```

```
) → DataFrame:
```

Create:

swap_month

using:

```
date_trunc(
```

```
    "month",
```

```
    col("swap_date")
```

```
)
```

Q.1

Q.2

Task 3 — Filter Valid Swaps — 2 Marks

```
def filter_valid_swaps(  
    df: DataFrame  
    ) → DataFrame:
```

```
    Keep rows where:
```

```
    energy_kwh >= 0
```

```
    and:
```

```
    battery_type IS NOT NULL
```

Task 4 — Monthly Battery Energy — 2 Marks

```
def monthly_battery_energy(  
    df: DataFrame  
    ) → DataFrame:
```

```
    Group by:
```

```
    battery_type
```

```
    swap month
```

Q.1

Q.2

Task 4 — Monthly Battery Energy — 2 Marks

```
def monthly_battery_energy(  
    df: DataFrame  
    ) → DataFrame:
```

Group by:

battery_type

swap_month

Calculate:

SUM(energy_kwh)

Alias:

total_energy_kwh

Required output:

battery_type

Q.1

Description

Q.2

Required output:

battery_type

swap_month

total_energy_kwh

Task 5 — Top Battery Type — 2 Marks

```
def top_battery_type(
```

```
    df: DataFrame
```

```
) → Tuple[str, float]:
```

Expected input contains:

battery_type

total_energy_kwh

The evaluator supplies this DataFrame directly.

Logic

- group by battery_type
- sum total_energy_kwh
- sort total descending
- battery type ascending for tie

Q.1

Description

Q.2

Logic

- group by battery_type
- sum total_energy_kwh
- sort total descending
- battery_type ascending for tie
- return:

(battery_type, total_energy)

Empty:

("", 0.0)

Tests

TC1 load_battery_swap_data

TC2 add_swap_month

TC3 filter_valid_swaps

TC4 monthly_battery_energy

TC5 top_battery_type

TC6 top_battery_type tie + empty